ДОМАШНЕЕ ЗАДАНИЕ

Обработка Frontend-вакансий в RustFS

MapReduce, SQL и сравнение производительности

Выполнил Дубовик Никита

Индивидуальный вариант 8

01 Хранилище Четыре папки и данные в hh-lake	**02 Подсчёт навыков** Навыки, пары и тройки двумя способами
03 Сравнение подходов Время, память и переход на Parquet	**04 Анализ вакансий** Зарплаты и сравнение трёх регионов

Что было сделано

Я развернул RustFS на четырёх папках и загрузил исходный CSV с ИТ-вакансиями. В этот раз использовал весь каталог hh, поэтому в хранилище находятся все 10 000 JSON. Для расчётов выбрал только вакансии своего варианта, программистов и разработчиков Frontend.

Данные	Количество	Размер, байт
Исходный CSV	229 388 строк	847 619 384
JSON из каталога hh	10 000 файлов	54 984 037
Отобранные Frontend-вакансии	6 057 строк	1 727 780

Запуск хранилища

```
.venv/Scripts/python.exe scripts/init_env.py
docker compose up -d
.venv/Scripts/python.exe -m frontend_lake.storage
```

Для опыта использовал RustFS 1.0.0 в Docker. Папки /data/vol1, /data/vol2, /data/vol3 и /data/vol4 подключены через каталог storage. S3 доступен на порту 9010, консоль на порту 9011.

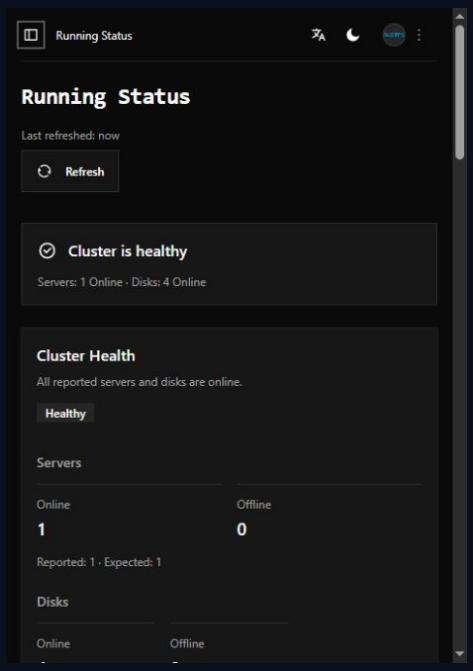


Рисунок 1. В консоли RustFS видны один доступный сервер и четыре доступные папки.

Все папки находятся на одном компьютере и общем накопителе. Такая схема подходит для учебного опыта, но не защищает от отказа физического диска.

Данные в хранилище

В RustFS создан бакет hh-lake. CSV загружен одним объектом в raw/it_vacancies_2024.csv. Каждый JSON сохранён отдельным объектом в raw/hh. Дополнительно я сохранил отобранные 6 057 вакансий в prepared/frontend_variant8.csv.

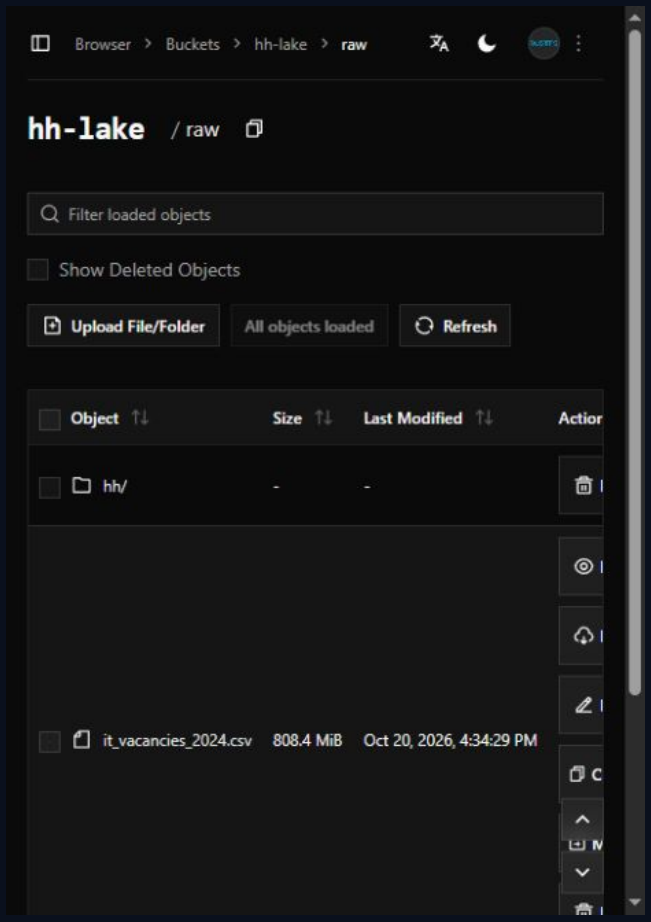


Рисунок 2. В бакете hh-lake видны исходный CSV размером 808,4 МиБ и папка hh с JSON.

Проверка загрузки

После загрузки я скачал CSV из S3 и сравнил SHA-256 с исходным файлом. Хеши совпали. Для всех 10 000 JSON проверены размеры объектов. Для 6 файлов дополнительно сравнено содержимое по SHA-256.

```
CSV SHA-256: совпал
JSON: размеры 10 000 из 10 000 совпали
JSON: SHA-256 выбранных файлов совпал (6)
```

Список объектов, размеры и хеши сохранены в output/manifest.json и output/storage_evidence.json. В JSON-архиве фильтру соответствуют 245 вакансий. Я не объединял этот архив с CSV, чтобы не посчитать одну вакансию дважды. Оба способа расчёта читают один и тот же CSV.

Как отобраны вакансии варианта 8

В задании указан вариант «Программист, разработчик Frontend». Дополнительного фильтра не было. Я оставил строки, где роль точно равна «Программист, разработчик», а в названии явно написано Frontend или его русское написание.

```
WHERE strip_whitespace(role) = 'Программист, разработчик'
  AND regexp_matches(lower(name),
  '(^|[^a-za-яё])(front[ -]*end|фронт[ -]*[еэ]нд)([^a-za-яё]|$)')
```

Фильтр учитывает Frontend, Front-end, Front end, Фронтенд и Фронтэнд в любом регистре. React developer без слова Frontend не включается. Frontend/Fullstack включается, потому что направление Frontend указано прямо. Отбор по роли и названию записан отдельно в Python и SQL.

Проверка	Результат
Строк в исходном CSV	229 388
Вакансий Frontend	6 057
Уникальных пар источник и ID	6 057
Вакансий без заполненных навыков	1 502
Максимальный размер стека	30
Период публикаций в исходном CSV	2023-12-19 ... 2024-09-14

Подготовка навыков

Я разделяю поле key_skills по точке с запятой, убираю пробельные символы по краям и привожу названия к нижнему регистру. SQL-функция strip_whitespace учитывает те же пробелы, табуляции и переносы строк, что и Python. Повторения внутри одной вакансии удаляю. Затем сортирую список и беру первые 5, первые 10 или все навыки.

Эти три размера нужны для сравнения количества сочетаний. Алфавитный порядок не означает важность навыка. Зарплаты и регионы я анализирую по полному стеку. В основных частотах react и react.js остаются разными метками. Проверка синонимов выполнена отдельно.

Два способа подсчёта

Собственный MapReduce

Программа открывает CSV через boto3 GetObject. Буфер чтения составляет 1 МиБ. После разбора CSV строки собираются в чанки по 1 000 записей. Так переносы строк в кавычках и границы UTF-8 не разрывают записи.

Для каждой подходящей вакансии Map создаёт навыки, пары и тройки. Локальный счётчик складывает частоты внутри чанка. Reduce объединяет счётчики всех чанков. Весь CSV в память не загружается, но память всё равно растёт с числом уникальных сочетаний.

Это моя одноузловая реализация модели MapReduce. Распределённый кластер вычислителей в этом опыте не разворачивался.

DuckDB

DuckDB читает тот же объект S3 через расширение httpfs. Запрос самостоятельно отбирает Frontend-вакансии и готовит списки навыков. UNNEST разворачивает список вместе с позицией каждого навыка.

```
SELECT a.skill, b.skill, c.skill, count(*)
FROM expanded a
JOIN expanded b ON a.rid=b.rid AND a.pos<b.pos
JOIN expanded c ON a.rid=c.rid AND b.pos<c.pos
GROUP BY a.skill, b.skill, c.skill
```

Позиция первого навыка должна быть меньше позиции второго, а позиция второго меньше позиции третьего. Так каждая неупорядоченная тройка получается один раз. Для пар используется соединение двух списков. В SQL нет вызовов Python и чтения готовых результатов MapReduce.

Как проверено совпадение

Я сравнил полный словарь сочетаний и частот, а также ID отобранных вакансий. Одних общих сумм было бы недостаточно. Все 27 результатов совпали с эталоном для своего размера стека. Хеши словарей сохранены в output/verification.json.

Частоты и рост числа сочетаний

Стек	Навыки	Пары	Тройки	Разных навыков	Разных пар	Разных троек
5	20 217	36 991	34 632	932	6 040	11 203
10	28 140	88 388	182 809	1 283	14 830	62 162
все	30 659	121 466	399 695	1 470	22 963	160 010

В первых трёх числовых столбцах показаны суммы частот. Если одинаковая пара встретилась в ста вакансиях, она даёт сто упоминаний. В остальных столбцах указано количество разных сочетаний.

Вакансия с m разными навыками даёт m одиночных навыков, m(m-1)/2 пар и m(m-1)(m-2)/6 троек. Поэтому добавление навыков особенно сильно увеличивает число троек.

Навыков в вакансии	Пар	Троек
5	10	10
10	45	120
20	190	1 140
30	435	4 060

При переходе от стека 5 к полному стеку число пар выросло с 36 991 до 121 466, а троек с 34 632 до 399 695. Для стека 5 троек меньше, чем пар, потому что часть вакансий содержит меньше пяти заполненных навыков.

Самые частые сочетания полного стека

Тип	Сочетание	Вакансий
Навык	javascript	3 189
Навык	typescript	2 341
Пара	javascript + typescript	1 655
Пара	javascript + react	1 413
Тройка	javascript + react + typescript	948
Тройка	css + html + javascript	682

Как измерены время и память

Для каждого способа и размера стека я сделал по 3 запуска в новом процессе Python. Всего получилось 27 попыток. Запуски шли последовательно, порядок способов менялся между повторениями.

Полное время включает запуск процесса, импорт библиотек, чтение из S3, отбор вакансий, подсчёт и запись результата. Именно его я использую для сравнения. Время внутри рабочего процесса до записи JSON сохранено отдельно в benchmark_raw.csv.

Память измерялась через psutil каждые 10 мс. Для процессов дерева берётся сумма их OS peak_wset. Это верхняя оценка одновременного пика, потому что максимумы разных процессов могли наступить в разное время. Она включает нативную память DuckDB. Память контейнера RustFS в неё не входит.

Среда опыта: Python 3.12.14, DuckDB 1.5.6, Docker 29.8.0. На компьютере 16 логических CPU и 63,9 ГиБ ОЗУ. DuckDB использует 4 потока с ограничением памяти 2GB.

Кэши ОС и RustFS я не сбрасывал. Поэтому результаты относятся к повторным запускам с прогретыми кэшами. На компьютере работали фоновые процессы и другие контейнеры. Показатель docker stats сохранён как отдельный снимок, а не как пик памяти хранилища.

Способ	Стек	Время, с	Разброс, с	Память, МиБ
MapReduce	5	10,706	10,634-11,389	93,7
DuckDB CSV	5	11,166	10,442-11,728	1 156,4
DuckDB Parquet	5	1,433	1,324-1,525	183,7
MapReduce	10	12,041	11,274-12,456	168,0
DuckDB CSV	10	11,612	11,300-11,631	1 156,3
DuckDB Parquet	10	1,865	1,839-1,900	200,6
MapReduce	все	12,327	11,653-13,334	318,4
DuckDB CSV	все	11,946	11,685-12,160	1 156,2
DuckDB Parquet	все	2,830	2,787-3,109	354,3

Время и память в таблице показаны как медианы 3 попыток. Разброс показывает минимальное и максимальное время. На CSV времена MapReduce и DuckDB близки, диапазоны трёх попыток перекрываются. Медиана памяти MapReduce составляет 318,4 МиБ, DuckDB на CSV 1 156,2 МиБ.

Все попытки профилирования

Способ	Стек	Попытка	Время, с	Память, МиБ
DuckDB CSV	все	1	11,946	1 156,2
DuckDB CSV	все	2	12,160	1 156,2
DuckDB CSV	все	3	11,685	1 156,4
DuckDB CSV	5	1	10,442	1 156,4
DuckDB CSV	5	2	11,728	1 155,9
DuckDB CSV	5	3	11,166	1 156,4
DuckDB CSV	10	1	11,300	1 156,3
DuckDB CSV	10	2	11,631	1 156,0
DuckDB CSV	10	3	11,612	1 156,4
DuckDB Parquet	все	1	3,109	354,1
DuckDB Parquet	все	2	2,830	354,3
DuckDB Parquet	все	3	2,787	354,9
DuckDB Parquet	5	1	1,324	183,6
DuckDB Parquet	5	2	1,525	184,1
DuckDB Parquet	5	3	1,433	183,7
DuckDB Parquet	10	1	1,865	199,7
DuckDB Parquet	10	2	1,839	201,1
DuckDB Parquet	10	3	1,900	200,6
MapReduce	все	1	12,327	318,4
MapReduce	все	2	13,334	320,2
MapReduce	все	3	11,653	317,8
MapReduce	5	1	11,389	92,6
MapReduce	5	2	10,634	94,4
MapReduce	5	3	10,706	93,7
MapReduce	10	1	11,274	174,4
MapReduce	10	2	12,041	167,5
MapReduce	10	3	12,456	168,0

Для каждой попытки сохранены команда, вывод программы и код завершения. Эти файлы находятся в output/logs. В output/benchmark_raw.csv лежат все измерения, в output/benchmark_summary.csv находятся сводные значения.

Оптимизация через Parquet

Я подготовил Parquet со сжатием ZSTD. В нём оставлены все исходные строки и 11 столбцов, которые нужны для расчётов. Длинные описания вакансий и другие ненужные поля исключены. Frontend заранее не отбирался, фильтр выполняется при каждом запросе.

Parquet создан из исходного CSV в S3 и загружен обратно в prepared/vacancies.parquet. Запрос считает те же навыки, пары и тройки. Полные результаты для трёх размеров стека совпали с CSV и MapReduce.

Полный стек	CSV	Parquet
Медиана времени, с	11,946	2,830
Медиана пика памяти, МиБ	1 156,2	354,3
Размер объекта, байт	847 619 384	7 424 981
Количество попыток	3	3

Время уменьшилось в 4,22 раза. Память клиента уменьшилась в 3,26 раза. Размер объекта стал меньше в 114,16 раза.

Цена подготовки

Создание и загрузка Parquet заняли 10,874 с. Это время не входит в последующие запросы. При экономии 9,116 с на запрос подготовка окупается примерно после 2 запросов полного стека.

Ускорение связано сразу с форматом, сжатием и исключением ненужных столбцов. Я не разделял вклад каждого фактора. Для одного запроса нужно учитывать время подготовки целиком. По памяти у MapReduce получилось 318,4 МиБ, у DuckDB на Parquet 354,3 МиБ.

Как сравнивается зарплата

Для сравнения я оставил только рублёвые зарплаты с явным признаком «на руки». Это currency=RUR и gross=false. Зарплаты до налога, другие валюты и неизвестный налоговый статус не смешиваются с этой выборкой.

Если заданы обе границы, использую середину диапазона. Если указана только одна граница, беру её как приближение. Пустые, неположительные, нечисловые и перевёрнутые диапазоны исключаю. Отдельного поля периода выплаты в данных нет, поэтому месячная трактовка остаётся допущением.

Показатель	Значение
Вакансий с подходящей зарплатой	1 851
Доля от всех Frontend-вакансий	30,6%
Общая медиана, руб.	140 000
Диапазонов с двумя границами	939
Их медиана, руб.	130 000
Только нижняя граница	624
Только верхняя граница	288

Для каждого сочетания сравниваю медиану вакансий со всеми его навыками и медиану остальных вакансий зарплатной выборки. Разница равна первой медиане минус вторая. В группе с сочетанием должно быть не меньше 20 вакансий, в группе без него не меньше 20.

Далее n показывает размер группы с сочетанием. Группа без него равна 1 851 минус n. Столбец «С» содержит медиану с навыками, «Без» содержит медиану без сочетания. Разница не доказывает причинного влияния навыка. На неё могут влиять опыт, регион и компания.

Я проверяю много пересекающихся сочетаний и не делаю поправку на множественные сравнения. Поэтому это поиск наблюдений в данных, а не доказательство статистической значимости. Медиана только закрытых диапазонов составляет 130 000 рублей вместо 140 000 рублей. Открытые границы влияют на результат.

Пары навыков и зарплата

Наибольшая положительная разница

Навыки	n	С, руб.	Без, руб.	Разница, руб.
rxjs + typescript	26	230 000	135 000	95 000
frontend + react.js	30	221 250	135 000	86 250
react.js + typescript	66	202 500	135 000	67 500
css + разработка по	21	205 000	138 750	66 250
angular + javascript	80	200 000	135 000	65 000

Наибольшая отрицательная разница

Навыки	n	С, руб.	Без, руб.	Разница, руб.
javascript + веб-программирование	28	67 500	140 000	-72 500
ajax + html	34	70 000	140 000	-70 000
jquery + кроссбраузерная верстка	25	70 000	140 000	-70 000
bootstrap + кроссбраузерная верстка	21	70 000	140 000	-70 000
css + jquery	52	75 000	140 000	-65 000

Самая большая положительная разница получилась для rxjs + typescript. Медиана выше на 95 000 рублей при n=26. Для javascript + веб-программирование медиана ниже на 72 500 рублей при n=28.

Эти результаты относятся к исходным меткам навыков. Медианы и размеры показанных групп я дополнительно проверил независимым SQL-запросом. Полный список находится в output/salary_associations.csv.

Тройки навыков и зарплата

Наибольшая положительная разница

Навыки	n	С, руб.	Без, руб.	Разница, руб.
react + react.js + typescript	29	250 000	135 000	115 000
frontend + react + react.js	26	250 000	135 000	115 000
frontend + react.js + typescript	24	250 000	135 000	115 000
react + reactjs + typescript	24	250 000	135 000	115 000
frontend + reactjs + typescript	20	250 000	135 000	115 000

Наибольшая отрицательная разница

Навыки	n	С, руб.	Без, руб.	Разница, руб.
ajax + css + html	22	65 000	140 000	-75 000
html + javascript + php	38	70 000	140 000	-70 000
css3 + html + кроссбраузерная верстка	37	70 000	140 000	-70 000
css3 + javascript + php	33	70 000	140 000	-70 000
ajax + html + javascript	31	70 000	140 000	-70 000

Самая большая положительная разница получилась для react + react.js + typescript. Медиана выше на 115 000 рублей при n=29. Для ajax + css + html медиана ниже на 75 000 рублей при n=22.

Эти результаты относятся к исходным меткам навыков. Медианы и размеры показанных групп я дополнительно проверил независимым SQL-запросом. Полный список находится в output/salary_associations.csv.

В верхней части есть react и react.js внутри одной тройки. Они обозначают одну технологию, поэтому такой результат нельзя считать сочетанием трёх независимых компетенций. Ниже я отдельно проверяю влияние синонимов.

Что меняется при объединении синонимов

Для дополнительной проверки я объединил react.js и reactjs в react, vue.js и vuejs в vue, angular.js в angularjs. Angular и AngularJS оставил разными технологиями. После замены снова удалил повторы внутри каждой вакансии.

Основные контрольные частоты MapReduce и SQL не меняются. Это отдельная проверка зарплатного анализа с теми же правилами отбора зарплат и порогом 20 вакансий.

Пары после объединения синонимов

Разница	Навыки	n	С, руб.	Без, руб.
95 000	rxjs + typescript	26	230 000	135 000
66 250	css + разработка по	21	205 000	138 750
65 000	angular + javascript	80	200 000	135 000
-72 500	javascript + веб-программирование	28	67 500	140 000
-70 000	ajax + html	34	70 000	140 000
-70 000	jquery + кроссбраузерная верстка	25	70 000	140 000

Тройки после объединения синонимов

Разница	Навыки	n	С, руб.	Без, руб.
70 000	angular + javascript + typescript	56	205 000	135 000
70 000	angular + css + typescript	20	205 000	135 000
65 000	mobx + react + typescript	28	200 000	135 000
-75 000	ajax + css + html	22	65 000	140 000
-70 000	html + javascript + php	38	70 000	140 000
-70 000	css3 + html + кроссбраузерная верстка	37	70 000	140 000

После объединения синонимов среди троек с наибольшей положительной разницей выделяется angular + javascript + typescript. Медиана составляет 205 000 рублей против 135 000 рублей у остальных вакансий, n=56.

У ajax + css + html медиана составляет 65 000 рублей против 140 000 рублей, n=22. Это наблюдение о составе вакансий. Из него не следует, что отдельный навык сам по себе снижает зарплату. Дополнительные таблицы сохранены в salary_associations_canonical.csv и salary_extremes_canonical.json.

Сравнение трёх регионов

Я выбрал три региона: Москва, Санкт-Петербург, Екатеринбург. Регион определяется точным значением area. Для удалённой вакансии город публикации может отличаться от места работы сотрудника.

Регион	Вакансий	С зарплатой	Доля, %	Медиана, руб.
Москва	2 774	696	25,1	175 000
Санкт-Петербург	932	310	33,3	145 000
Екатеринбург	241	115	47,7	115 000

Зарплаты рассчитаны по той же рублёвой выборке на руки. Доля показывает, у какой части вакансий региона есть подходящая зарплата. Это важно, потому что покрытие различается почти вдвое.

Навык	Москва	Санкт-Петербург	Екатеринбург
javascript	1330 (47,9%)	482 (51,7%)	147 (61,0%)
typescript	1141 (41,1%)	370 (39,7%)	102 (42,3%)
react	931 (33,6%)	292 (31,3%)	79 (32,8%)
git	616 (22,2%)	257 (27,6%)	71 (29,5%)
html	474 (17,1%)	175 (18,8%)	88 (36,5%)
css	376 (13,6%)	148 (15,9%)	79 (32,8%)

Частоты навыков посчитаны среди всех Frontend-вакансий региона. Москва даёт 2 774 вакансии против 932 в регионе Санкт-Петербург и 241 в регионе Екатеринбург. Поэтому для сравнения востребованности я смотрю на доли, а не только на количество упоминаний.

В регионе Екатеринбург JavaScript указан в 61,0% вакансий, в регионе Москва в 47,9%. Разница составляет 13,1 процентного пункта. Доля вакансий с опытом от 3 до 6 лет составляет 37,8% и 58,5% соответственно. Разный состав опыта может частично объяснять разницу зарплат.

В знаменатель долей входят и вакансии с пустыми навыками. Поэтому показатели отражают также полноту описаний. Обе региональные таблицы сверены с отдельными SQL-запросами из sql/regions.sql.

Итоги и воспроизведение

MapReduce и DuckDB дали одинаковые частоты навыков, пар и троек для всех трёх стеков. На CSV времена MapReduce и DuckDB близки, диапазоны трёх попыток перекрываются. Переход DuckDB на Parquet ускорил полный расчёт примерно в 4,22 раза с учётом указанных условий опыта.

Для зарплатного сравнения важно разделять валюту, налоговый статус и открытые диапазоны. После объединения синонимов верхние сочетания меняются, поэтому качество меток влияет на выводы. Более высокая зарплата в Москве также сопровождается другой структурой требований к опыту.

Основные команды

```
.venv/Scripts/python.exe scripts/init_env.py
docker compose up -d
.venv/Scripts/python.exe -m frontend_lake.storage
.venv/Scripts/python.exe -m frontend_lake.pipeline
.venv/Scripts/python.exe -m frontend_lake.analytics
.venv/Scripts/python.exe scripts/evidence.py
.venv/Scripts/python.exe -m frontend_lake.report
.venv/Scripts/python.exe scripts/render_pdf.py
.venv/Scripts/python.exe scripts/publish_results.py
.venv/Scripts/python.exe scripts/check_submission.py --remote
```

Команды запускают Python из виртуального окружения проекта. Исходный код находится в frontend_lake, SQL-запросы в sql. Все 27 попыток и выводы команд сохранены в output. Частоты для стеков 5, 10 и всех навыков загружены в hh-lake/results/variant8/counts.